

Week 1 Task Answers and Discussion Points

This document answers every Week 1 task in the assignment, in task order. It describes the current Spark implementation and distinguishes measured evidence from proposed extensions. The companion five-page design report summarizes the design; this document provides the detailed transformation inventory and discussion answers.

Task 1 Study the Data

Taxi trips

The entity is one yellow-taxi trip. The supplied Parquet has no guaranteed natural primary key. Silver uses `trip_id`, a SHA-256 hash of `vendor_id`, `pickup_datetime`, `dropoff_datetime`, `pickup_location_id`, `dropoff_location_id`, `fare_amount` and `trip_distance`. This is a practical duplicate policy: identical values can describe distinct physical trips, and a fare correction changes the hash. It is not a correction-stable business identifier.

Join attributes are `pickup_location_id` and `dropoff_location_id` for zones, and the containing UTC pickup or dropoff hour for context. Temporal attributes include both journey timestamps, retained local timestamps, `pickup_date` and derived year, month, day and hour. Categories include vendor, payment type, rate code, store-and-forward flag and location IDs. Growth means additional trip records and date partitions; the schema need not grow with each release.

Weather

The entity is one hourly observation in the supplied weather series. The source key is year, month, day and hour; `weather_observation_id` concatenates these components. This assumes one series and at most one observation per source hour. Add `station_id` or `source_id` when multiple series are introduced. UTC `observation_datetime` is the temporal join key. Source calendar fields and `observation_date` are also temporal. Weather condition code, cloud cover and source flags are categorical or coded attributes. New hours increase row count; new stations multiply that growth.

Air quality

The entity is one hourly monitor occurrence measurement of PM2.5. The operational key hashes `state_code`, `county_code`, `site_number`, `poc` and `observation_datetime`. POC distinguishes instrument occurrences at a site. The general EPA identity also needs `parameter_code`; its omission relies on the input being the supplied single-parameter 88101 file. A multi-pollutant extension must add the parameter and validate units.

Join attributes are derived borough and UTC `observation_datetime`, after aggregation. Source Date Local, Time Local, Date GMT and Time GMT are temporal; Silver retains UTC date and hour. County, borough, parameter, units and instrument/method codes are categorical. Growth comes from new hours, sites, instruments and potentially parameters. The national source is large even though NYC Silver is small.

Taxi zone lookup

The entity is one location reference entry. LocationID becomes the integer primary key `location_id` and joins to both trip endpoint IDs. There is no temporal attribute in this snapshot. Borough, zone and `service_zone` are descriptive categorical fields. Growth is occasional reference additions or corrections; historical boundary versions would require effective dates and a changed key.

Observed input and accepted counts

The current execution JSON records input, accepted, rejected and out-of-scope counts per dataset. The validation record identifies the full-data run used for the updated implementation; older evidence is labelled historical.

Task 2 Design Your Storage Architecture

Directory structure and table organization

Original Parquet and CSV files remain in data/. Under storage/delta, bronze/raw_taxi_trips, raw_weather, raw_air_quality and raw_taxi_zones hold separate Delta snapshots. Silver contains clean_taxi_trips, clean_weather, clean_air_quality, clean_taxi_zones and integrated_taxi_trips. Every table has its own data files and Delta transaction log; there is no external catalog registration requirement. Readers use Delta paths.

Rejected records append to storage/delta/quarantine/<dataset>. Successful ingestion audit rows append to storage/delta/metadata/ingestion_runs. JSON run summaries go to storage/metrics. Benchmark tables live under storage/delta/benchmark/<strategy>/<run_id>, preserving earlier experiments. Full platform runs overwrite Bronze and Silver and append quarantine and metadata history. Publication is atomic per Delta table, not across the entire pipeline.

Naming conventions and partitioning

Silver fields use lower snake_case with meaningful unit suffixes where defined. Table names distinguish raw_, clean_ and integrated_ roles. The active YAML partitions taxi and integrated tables by pickup_year and pickup_month, derived from New York local pickup time. Weather, cleaned air quality and zones are unpartitioned. Bronze is unpartitioned. Direct class construction has a daily taxi/integration fallback, but the normal runner explicitly supplies the monthly YAML configuration.

Which datasets are conceptually lookup tables

Taxi zones are a reusable spatial dimension. Weather is a temporal lookup at hour grain. Site-to-borough and citywide hourly air aggregates are temporal-spatial lookups; the underlying monitor readings are observation facts. Taxi trips remain the large fact table. Calling context a lookup describes its role in integration, not an assumption that all future observations stay small.

Which datasets should not be partitioned and why

The 264 accepted zone entries, 8,784 weather rows and 51,855 NYC air readings are small enough that fine partitions would add directories, metadata and small files without useful scan reduction for the current joins. Nationwide Bronze air data is larger, but is processed as a full snapshot in this implementation. Partitioning it should follow an actual regional or time-filtered Bronze workload rather than copy the Silver policy automatically.

Which datasets require different strategies and when partitioning is harmful

Large taxi facts benefit from evaluating date layouts; stable zones do not. At larger scale, sensor observations may need date or region organization independently of taxi partitions. Partitioning is harmful when each slice is tiny, the key has high cardinality, values are skewed, or queries scan all partitions anyway. It can increase file-open, metadata and scheduling costs and reduce writer parallelism. Partition columns must also be available in query predicates; filtering pickup_date does not itself prove pruning of pickup_year/pickup_month partitions.

What changes at twenty times the volume

If trips per day rise twentyfold, daily partitions may become large enough to justify their overhead. If the date range grows twentyfold with unchanged daily density, monthly partitions may remain preferable. Measure both cases before migrating. Increase worker capacity and writer parallelism, compact small files and set workload-based target file sizes. The integrated writer repartitions only by partition keys, which can constrain parallelism for a short date range. Use multiple writers per large partition when necessary. Consider shared object storage and a catalog for multi-user operation, and recheck context broadcast sizes. These are proposed extensions, not features demonstrated by the local benchmark.

Task 3 Build a Generic Ingestion Framework

How the required ingestion steps are implemented

BaseDatasetIngestor reads Parquet or header-based CSV, checks required columns, writes Bronze, applies configured renames and tolerant casts, invokes scope filtering and domain transformations, evaluates quality rules, deduplicates, writes quarantine and Silver, and records metadata. CSV starts as strings so malformed numerical values can be identified. Structurally malformed CSV uses FAILFAST; such file-level failures stop the run rather than becoming row-level quarantine records.

Required-column validation detects missing fields but does not constitute a strict complete schema contract. Extra columns are accepted. Column normalization rejects collisions. Cast failures in configured fields are carried as `invalid_type:<column>`, even if a later transformation supplies a default. False or null SQL validity predicates reject the row. Missing key fields also reject it. Valid duplicate groups use deterministic row ranking; one row survives and other occurrences receive `duplicate_key`.

Which components are generic and reusable

Readers, required-column checks, naming, casts, named SQL validation, duplicate ranking, Delta persistence, rejection annotation, row reconciliation and metrics are reusable. Configured format, key and rule definitions keep these operations shared. The core asserts `initial_records = valid_records + rejected_records + out_of_scope_records`. Rejected counts include duplicates; `invalid_records` excludes duplicates.

Which components remain dataset specific and why

Taxi normalization interprets local wall time, derives durations and constructs the trip hash. Weather assembles timestamps from calendar fields and chooses its Silver projection. Air quality selects NYC, maps counties to boroughs and builds monitor identities. Zones trim reference labels. These operations encode source meaning that a generic cast or rename cannot infer safely.

How transformation rules are defined and maintained

`config/platform_config.yaml` is the active configuration: datasets specifies format, `required_columns`, `rename`, casts, key, and named SQL validation predicates, with per-dataset quality rules and timezones directly integrated. Threshold placeholders are substituted into validity predicates. Python subclasses maintain derived columns and domain projections. Changes should update configuration, schema version, this transformation inventory and relevant regression fixtures together. Configuration is the single source of source contracts; CSV loading reads strings before explicit casts. Constructor paths and versions are required, and every source/rule/output schema is resolved before table publication.

How metadata is managed

Each successful ingestion appends a Delta audit row with dataset, format, initial/valid/rejected/duplicate/invalid/out-of-scope counts, duration, schema version, run ID, paths, partition columns, timezone and `completed_at`. Duration excludes the audit write itself. Quarantine adds `_run_id`, `_schema_version`, `_rejected_at` and rejection reasons. The runner writes a latest JSON summary and timestamped copy containing runtime, effective configuration, status, ingestion, integration and optional benchmark metrics. Failed executions can appear in runner JSON, but the Delta ingestion audit is not a comprehensive failed-run monitoring system. Schema version is manually maintained, not automatically inferred.

How duplication and maintenance effort are reduced

All four ingestors use the same lifecycle and validity engine. A changed threshold normally requires a YAML change instead of four separate control flows. Domain logic remains short and explicit. Regression fixtures can test shared rules once and add focused timezone, missing-context and identity cases for particular sources.

What changes for twenty new datasets

Add source paths, output paths, schema specifications, quality rules, keys, timezones and partition choices. Add and register transformation subclasses when semantics differ, plus representative fixtures. The base class takes an explicit specification. Register new domain ingestors and update the runner contract when expanding beyond the current four datasets. Add readers for new formats, explicit schema compatibility policy and scheduling/resource controls as needed. Integration must be extended separately for new relationships; ingestion registration alone does not enrich taxi rows.

Task 4 Design a Common Data Model

Timestamp format naming missing values and types

Canonical instants use Spark TimestampType in a UTC session; an interchange representation is YYYY-MM-DDTHH:mm:ssZ. Delta stores typed timestamps rather than that text format. Taxi pickup_date and pickup_year/month/day/hour retain New York calendar meaning, while context observation_date uses UTC. Fields use lower snake_case. IDs and calendar categories are integer where configured; measurements and monetary values use double; labels and hash IDs use string; dates use DateType and match indicators use boolean. Double is suitable for this analytical prototype but decimal would be preferable for exact monetary accounting.

Missing observations remain null. Taxi passenger_count defaults to 1, tip_amount and tolls_amount to 0; these are explicit assumptions that can bias summaries. Malformed non-null cast values are still rejected. Missing pickup/dropoff labels become Unknown during integration; missing service zones can remain null. Required validation fields cannot be null. No rainfall or pollution value is fabricated to fill missing context.

The integrated model retains pickup_pm25_borough, a nullable double borough-hour concentration, and pickup_pm25_citywide, a nullable double mean across available NYC sites at that UTC hour. The citywide value is identical for every pickup in the same hour, and null if no sites report. Concentrations retain the source units, micrograms per cubic meter in the supplied file. Missing borough values are never replaced inside pickup_pm25_borough. The existing pickup_pm25 convenience column still selects borough then citywide, with pickup_pm25_source recording borough, citywide or missing. Corresponding dropoff columns apply the same definitions at the dropoff hour.

Complete transformation inventory

Generic transformations: preserve source files; sanitize Bronze column spaces/parentheses/semicolons and add _ingestion_timestamp; map semantic field names, otherwise normalize to snake_case; reject duplicate normalized names; require and apply configured standardized cast fields; retain cast-error names; apply dataset scope; run domain rules; reject invalid keys/values; deduplicate valid key groups; remove internal validation arrays from accepted output; append audit fields to quarantine. Full raw field mappings and casts are enumerated in config/platform_config.yaml.

Taxi: VendorID becomes vendor_id; tpep pickup/dropoff fields become pickup_datetime/dropoff_datetime; PULocationID and DOLocationID become pickup_location_id/dropoff_location_id. RatecodeID becomes rate_code_id and Airport_fee becomes airport_fee; other mapped fare/category fields use snake_case. Cast vendor, passenger and location IDs to int, and distance/fare/total/tip/tolls to double. Retain parsed source times as pickup_local_datetime/dropoff_local_datetime; convert them from America/New_York to UTC. Derive local pickup_date/year/month/day/hour. Compute duration from UTC epoch differences in minutes, rounded to two decimals. Apply the three defaults described above and hash the seven identity fields listed in Task 1. Remaining taxi columns retain source types unless explicitly cast.

Weather: build an hourly timestamp from year/month/day/hour, parse tolerantly and convert from configured source timezone to UTC. Derive observation_date and the source-calendar observation ID. Rename temp, rhum, prcp, wspd, pres, cldc and coco to temperature_c, relative_humidity_pct, precipitation_mm, wind_speed_kmh, air_pressure_hpa, cloud_cover_oktas and weather_condition_code. Cast calendar and codes to int and main measurements to double. The Silver projection retains those measurements, time fields and ID; wind direction, snow depth, gusts and provenance flags stay in Bronze. No unit conversion or imputation is applied to retained weather measurements.

Air: normalize source names and cast state/county/site/parameter/POC to int, latitude/longitude and PM2.5 to double. Keep state 36 and counties 5, 47, 61, 81, 85; unknown geography can reach validation for rejection rather than silently disappear. Construct UTC time from Date GMT plus Time GMT, derive UTC date/hour, and map counties to Bronx, Brooklyn, Manhattan, Queens and Staten Island respectively. Hash monitor components and UTC time. Retain ID, time/date/hour, geography, site/POC, PM2.5, units and coordinates. Source local timestamps, parameter and method metadata remain in Bronze. No concentration conversion is implemented; the file is assumed to contain the intended PM2.5 parameter and compatible units.

Zones: rename LocationID/Borough/Zone, cast location_id to int and descriptive fields to string, then trim borough, zone and service_zone. Silver retains those four fields. Integration and aggregation transformations are detailed in Task 5.

Validation rules and limitations

Taxi: valid round-trip timezone conversion; rounded duration >0.1 and <=1440 minutes; local pickup in January-March 2024; fare 0-5000, distance 0-500, passenger count 0-10; finite nonnegative total/tips/tolls; non-null endpoint IDs. Weather: parseable timestamp, 2024 calendar bounds, temperature -40 to 60, humidity 0-100, finite nonnegative wind and observed precipitation, finite positive observed pressure, cloud cover 0-8 and condition code 1-27. Air: UTC year 2024, PM2.5 0-1000 and present borough/state/county/site/POC. Zones: nonblank borough and non-null key. Every dataset also rejects malformed configured casts and duplicates. There is no complete reference-integrity, unit or all-field range validation; unknown taxi location references survive with Unknown labels.

Task 5 Build the Integration Pipeline

Output grain and required enrichments

integrated_taxi_trips contains one row per accepted trip, retaining trip_id and taxi attributes. Required additions are pickup_temperature_c, pickup_humidity_pct, pickup_precipitation_mm, pickup_wind_speed_kmh, pickup_pressure_hpa, pickup_cloud_cover, pickup_weather_code, pickup_pm25, pickup_zone, pickup_borough, dropoff_zone and dropoff_borough. The implementation also adds service-zone labels, selected dropoff weather and dropoff PM2.5 as extensions beyond the required pickup context.

How an hourly weather observation is associated with a trip

Convert taxi time to UTC and truncate to the containing hour, then left-join weather on exact observation_datetime. For example, 2024-01-15 08:37 New York becomes 13:37 UTC and joins 13:00 UTC. This assumes an hour-labelled observation represents that hour. It avoids assigning the next hour merely because it is closer, but does not model within-hour variation. The supplied weather CSV has no timezone declaration; datasets.weather.timezone = UTC remains an assumption that must be confirmed with the export source. A successful join does not validate that assumption.

How hourly air measurements are associated with a trip

First average collocated instruments within each physical site/hour. Average site means within borough/hour and separately across all available NYC sites/hour; round both means to two decimals. Parallel left joins attach pickup_pm25_borough by borough and containing UTC hour, and pickup_pm25_citywide by hour alone. Keep both estimates independently even when a borough match exists. Dropoff uses the same logic at its time and borough. Equal site weighting prevents sites with more instruments from receiving disproportionate weight; it is not population, borough or area weighting. The fallback convenience column remains available separately.

How missing observations are handled

Every enrichment join is a left join. A missing weather hour leaves measurements null and sets pickup_weather_matched false. pickup_precipitation_missing distinguishes missing precipitation even in a matched weather row. Missing borough PM2.5 stays null in pickup_pm25_borough. The independent citywide field remains available where any NYC sites report that hour; if none report, it is null too. Only pickup_pm25 applies borough-to-citywide fallback, and pickup_pm25_source records the choice. Corresponding dropoff fields follow the same policy. No interpolation, nearest-hour search or forward fill is used. Unknown reference labels preserve the trip for aggregate totals.

How row preservation is verified

Zone IDs and observation keys are deduplicated upstream. Before publishing, the pipeline checks that source and integrated trip IDs are non-null and unique, and performs anti-joins in both directions to enforce exact ID-set equality. This detects lost rows, fanout and substitutions that a simple row-count comparison could miss. Only after those assertions does it overwrite the integrated Delta table. Required identity validation adds execution cost but protects the declared one-trip grain.

Limitations and trade offs

One weather series represents a spatially diverse city. Borough air means hide local variation and changing station availability. The supplied Q1 PM2.5 file has usable readings in the Bronx, Brooklyn and Queens, but none in Manhattan or Staten Island. This describes the supplied data, not the existence of monitoring stations. The citywide value averages available sites in the three covered boroughs, not all five boroughs equally.

reports/week1_validation.md records 929,709 borough matches and 8,464,621 pickups without a matching borough-hour estimate, or 90.1%, in the earlier run; it is not newly measured coverage of the separate columns.

Use pickup_pm25_citywide for a consistent citywide demand-versus-pollution series. For borough comparisons, use pickup_pm25_borough and report missing borough/hour coverage rather than silently interpreting the convenience fallback as local pollution. Neither series is individual exposure or causal evidence. Estimating Manhattan conditions requires additional observations or a spatial model validated against withheld sites. Use the current execution JSON for integration timing and coverage; historical values are not substituted for new measurements.

Hourly joins assume timestamps label comparable intervals. DST-invalid taxi times are rejected by round-trip validation; ambiguous autumn wall times follow Spark resolution and are outside the Q1 sample. EPA GMT fields avoid treating its local standard-time timestamps as New York daylight time. Broadcast joins suit current small context tables but must be reassessed as sources and cities grow. Exact ID checks and materializing the wide output consume additional compute and storage. Multi-table consistency, incremental refresh and historical zone boundaries are future work.

Task 6 Benchmark Your Design

Storage strategies and measurements

The implementation evaluates three layouts, exceeding the requirement for two: A unpartitioned, B partitioned by local pickup_date, and C by local pickup_year plus pickup_month. All operate on clean taxi facts derived separately from the same raw Parquet. It records ingestion duration, whole-directory bytes, Parquet data bytes, Parquet file count, total generated-file count, query samples, means, medians, sample standard deviations and physical plans. The JSON retains all measurements; the generated report presents the comparison tables.

Each layout timer includes raw loading, schema checks, standardization, transformations, disk materialization, validation, deduplication, equal writer redistribution and the clean Delta write. Bronze, quarantine, metadata writes, post-write row verification and directory inspection are excluded. This measures comparative raw-to-clean taxi ingestion, not the whole four-source platform duration. New run-ID directories prevent old files from contaminating storage measurements.

Required query definitions

Trips per borough groups by pickup borough and counts rows. Average trip duration per day groups by local pickup_date and averages the precomputed rounded trip_duration_minutes. Average fare per borough groups by pickup_borough and averages fare_amount. Borough queries left-join clean_taxi_zones on pickup_location_id and keep missing matches under Unknown. Optional Q4 selects February 14; Q5 selects February using year/month predicates. The same SQL is executed on every storage layout.

Benchmark results and interpretation

The canonical benchmark_report.md contains the measured tables and names the successful raw-input JSON used to generate them. It is the single numerical benchmark source; task answers do not maintain a second transcription. The runner compares three layouts using the same five queries and verifies equivalent results.

One ingestion trial per layout in fixed order and warm repeated queries limit conclusions. Judge query means alongside variability; do not infer pruning from timing alone. JSON retains EXPLAIN FORMATTED plans for inspection. Directory sizes include Delta metadata; the measured file counts do not imply a specific number of active writers. Monthly partitions remain a workload-dependent default.

How to reproduce and what to improve

Run python scripts/run_week1.py in the configured environment. A successful benchmark run writes fresh metrics and regenerates the Markdown report. The current YAML requests eight shuffle/writer partitions; do not assume it recreates the historical file counts. Record CPU, RAM, disk, Java, effective Spark settings and input hashes with new results. Repeat ingestion in randomized orders, include cold-cache trials where feasible, retain EXPLAIN FORMATTED output and measure scanned bytes before making stronger scalability claims. No twentyfold-scale experiment has been performed in the evidence available here.

Evidence map

Requirements: assignment_full_text.txt, Week 1 Tasks 1-6. Implementation: config/platform_config.yaml; src/common; src/ingestion; src/integration/pipeline.py; src/storage/benchmark.py; scripts/run_week1.py. Current measurements: reports/week1/benchmark_report.md and its named JSON artifact. Historical evidence is retained separately in reports/week1/evidence. The README gives executable commands and output semantics.